

SECTION II

PROGRAMMING THE GX-100/200 GRAPHIC DISPLAY FAMILY

2.0 OVERVIEW

2.1 FUNCTIONALLY - IMPORTANT HARDWARE

2.2 BASIC RESPONSIBILITIES OF THE PROGRAMMER

2.3 THE INSTRUCTION SET

2.3.0 The Set

2.3.1 The Philosophy Behind the Set

2.4 THE INSTRUCTIONS IN DETAIL

2.4.1 Data Instructions

2.4.2 Control Instructions

2.4.3 Address Instructions

2.4.4 The Single-bit Parameters

2.5 EXAMPLES

2.6 NOTES

11/8/72

2

JMB

FORMAT

1.0.□□□□□ XXXXXX

new ¶ : double space

□□□□□ 1.1.0□□□□□ XXXXXX

1.1.1□□□□□ XXXXXX

□□□□□ 1.1.1.0□□□□□ XXXXXX

□□□□□ 1.1.1.1.0□□□ www www

TABLE 4.0

INTERNAL REGISTERS RELEVANT TO PROGRAMMING

			# Bits
1	A	Additive-write Flag	1
2	C	Display Channel Reg.	8
3	DTA	Data Reg.	8
4	E	Element Address (or Displacement) Reg.	10
5	ECR	Element Current-value Reg.	10
6	EGR	Element Greater-value Reg.	10
7	ELR	Element Lesser-value Reg.	10
8	EXR	Element Index (i.e., "biasing") Reg.	10
9	F	Fixed-point Graphics Flag	1
10	H	Double-height Flag	1
11	I	Indexed-addressing Flag	1
12	L	Line Address (or Displacement) Reg.	10
13	LCR	Line Current-value Reg.	10
14	LGR	Line Greater-value Reg.	10
15	LLR	Line Lesser-value Reg.	10
16	LXR	Line Index (i.e., "biasing") Reg.	10
17	S	Sub-channel Reg.	4
18	T	Block Termination Flag	1
19	W	Double-width Flag	1
20	B	Background Flag	1
21	M	Control Mode Reg.	3

2.2 RESPONSIBILITIES OF THE PROGRAMMER

To initialize the display system the programmer must consider the following factors:

- 1) Mode: 1 of 8, to be ^{set} via an LCM instruction. (Refer to 2.4.2.6.)
- 2) Channel Accessing: SDC & SSC instructions must be properly configured and output to access the desired refresh sub-channel(s). (Refer to 2.4.2.1 and 2.4.2.2.)
- 3) Addressing: The eight address instructions must be used to establish the initial address -- (ECR) and (LCR). (Refer to 2.4.3.)
- 4) Data: If GX-100/200 is equipped with the high speed Input Buffer, a choice has to be made whether to pack data 2 bytes/word in a block defined by a BLK instruction, or to output data one byte at a time using LXD instructions. This choice has repercussions in the construction of data files. (Refer to 2.4.1.)
- 5) 1-bit Control Flags: The six single-bit control flags (Indexing, Background, Additive, Height, Width, & Fixed-point Graphics) must be considered every time an LCM instruction is output. These parameters may have a profound effect on the construction of the display image. (Refer to 2.4.4.)

The formal examples (2.5) provide detailed data files (with descriptions) used to produce a number of types of display images, all featuring the above factors.

After initializing the system, the programmer's next responsibility is the ordering of instruction sequences, the key to which lies in a thorough acquaintance with the "execute" instruction(s) appropriate to each of the eight control modes. Only after the "execute" instruction has been received and decoded will the incipient operation be executed. Thus it is up to the programmer to

2.2 (CTD)

ensure that all parameters and addresses have been properly set before issuing an "execute" instruction. Table 4.1 details these considerations.

RAMTEK GX-100/200 INSTRUCTION SET

		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Load & Execute Data	LXD	0	0	0	0	0	0	-	-	D	D	D	D	D	D	D	D
Block Transfer	BLK	0	0	0	0	0	1	T	R*	N	N	N	N	N	N	N	N
Select Display Channels	SDC	0	0	0	0	1	0	C* 9	C* 8	C 7	C 6	C 5	C 4	C 3	C 2	C 1	C 0
Select Sub-Channels	SSC	0	0	0	0	1	1	-	-	Z* 3	Z* 2	Z* 1	Z* 0	S 3	S 2	S 1	S 0
No Operation	NOP	0	0	0	1	0	0	-	-	-	-	-	-	-	-	-	-
Erase	ERS	0	0	0	1	0	1	-	-	-	-	-	-	-	-	-	-
Interrupt	INT	0	0	0	1	1	0	-	-	-	-	-	-	-	-	-	-
Load Control Mode	LCM	0	0	0	1	1	1	/	/	/	/	/	/	/	/	/	/
0) Alpha-Numeric Data								I	B	A	-	H	W	-	0	0	0
1) Transverse Data								I	B	A	-	H	W	-	0	0	1
2) Raster Data								I	B	A	-	H	W	-	0	1	0
3) Complex Data								I	B	-	-	H	W	-	0	1	1
4) Graphic Vector								I	B	-	-	-	-	F	1	0	0
5) Graphic Plot								I	B	-	-	-	-	F	1	0	1
6) Graphic Cartesian								I	B	-	-	-	-	-	1	1	0
7) Graphic Element								I	B	-	-	-	-	-	1	1	1
Load Element Relative	LER	0	0	1	0	0	0	E	E	E	E	E	E	E	E	E	E**
Load Element 1	LE1	0	0	1	0	0	1	E	E	E	E	E	E	E	E	E	E**
Load Element 2	LE2	0	0	1	0	1	0	E	E	E	E	E	E	E	E	E	E**
Load Element Index Reg.	LEX	0	0	1	0	1	1	E	E	E	E	E	E	E	E	E	E**
Load Line Relative	LLR	0	0	1	1	0	0	L	L	L	L	L	L	L	L	L	L**
Load Line 1	LL1	0	0	1	1	0	1	L	L	L	L	L	L	L	L	L	L**
Load Line 2	LL2	0	0	1	1	1	0	L	L	L	L	L	L	L	L	L	L**
Load Line Index Reg.	LLX	0	0	1	1	1	1	L	L	L	L	L	L	L	L	L	L**
Load Table Addr.	LTA	1	1	1	0	0	0	0	0	G	G	G	G	C	C	C	C
Load Table Data	LTD	1	1	1	1	B	B	B	B	G	G	G	G	R	R	R	R

Legend to the Instruction Set:

A Additive-write Flag

B. Background-select Flag

C. Display Channel Bit

D. Data Bit

E. Element Add (or Displacement) Bit

F. Fixed-point Graphics Flag

H. Double-height Flag

I. Double-height Flag

L. Line Addressing Flag

N. Block Length Bit

R. Reverse Packing Flag

S. Sub-Channel Bit

T. Block Termination Flag

W. Double-width Flag

Z. Forced-Zero Bit

* Applies to GX-100A only

** Not used for low resolution systems.

RAMTEK GX-100/200 INSTRUCTION SET

Load & Execute Data	LXD	0	0	0	0	0	0	-	-	D	D	D	D	D	D	D	D
Block Transfer	BLK	0	0	0	0	0	1	T	-	N	N	N	N	N	N	N	N
Select Display Channels	SDC	0	0	0	0	1	0	-	-	C	C	C	C	C	C	C	C
Select Sub-Channels	SSC	0	0	0	0	1	1	-	-	-	-	-	-	S	S	S	S
No Operation	NOP	0	0	0	1	0	0	-	-	-	-	-	-	-	-	-	-
Erase	ERS	0	0	0	1	0	1	-	-	-	-	-	-	-	-	-	-
Interrupt	INT	0	0	0	1	1	0	-	-	-	-	-	-	-	-	-	-
Load Control Mode	LCM	0	0	0	1	1	1	/	/	/	/	/	/	/	/	/	/
0) Alpha-Numeric Data								I	B	A	-	H	W	-	0	0	0
1) Transverse Data								I	B	A	-	H	W	-	0	0	1
2) Raster Data								I	B	A	-	H	W	-	0	1	0
3) Complex Data								I	B	-	-	H	W	-	0	1	1
4) Graphic Vector								I	B	-	-	-	-	F	1	0	0
5) Graphic Plot								I	B	-	-	-	-	F	1	0	1
6) Graphic Cartesian								I	B	-	-	-	-	-	1	1	0
7) Graphic Element								I	B	-	-	-	-	-	1	1	1
Load Element Relative	LER	0	0	1	0	0	0	E	E	E	E	E	E	E	E	E	E
Load Element 1	LE1	0	0	1	0	0	1	E	E	E	E	E	E	E	E	E	E
Load Element 2	LE2	0	0	1	0	1	0	E	E	E	E	E	E	E	E	E	E
Load Element Index Reg.	LEX	0	0	1	0	1	1	E	E	E	E	E	E	E	E	E	E
Load Line Relative	LLR	0	0	1	1	0	0	L	L	L	L	L	L	L	L	L	L
Load Line 1	LL1	0	0	1	1	0	1	L	L	L	L	L	L	L	L	L	L
Load Line 2	LL2	0	0	1	1	1	0	L	L	L	L	L	L	L	L	L	L
Load Line Index Reg.	LLX	0	0	1	1	1	1	L	L	L	L	L	L	L	L	L	L

Legend: to the Instruction Set

A Additive-write Flag
B Background-select Flag

C Display Channel Bit

D Data Bit

E Element Add. (or Displacement) Bit

F Fixed-point Graphics Flag

H Double-height Flag

I Indexed-addressing Flag

L Line Address (or Displacement) Bit

N Block-Length Bit

S Sub-Channel Bit

T Block Termination Flag

W Double-width Flag

TABLE 4.1

INSTRUCTION SEQUENCING

MODE	INSTRUCTIONS	REQUIRED	
		OPTIONAL (No specific order implied)	First "Execute"
0	Alphanumeric Data	LE1/LE2/LER, LL1/LL2/LLR	- LXD*
1	Transverse Data	LE1/LE2/LER, LL1/LL2/LLR	- LXD
2	Raster Data	LE1/LE2/LER, LL1/LL2/LLR	- LXD
3	Complex Data	LE1/LE2/LER, LL1/LL2/LLR	- LXD
4	Graphic Vector	LE1/LE2/LER, LL1 <i>FE1 uses FE2 & LLR</i>	LE1/LE2/LER LL2/LLR
5	Graphic Plot	LE1/LE2/LER, LL1	- LL2/LLR
6	Graphic Cartesian	LE1/LE2/LER, LL1	LE1/LE2/LER LL2/LLR
7	Graphic Element	LE1/LE2/LER	- LL2/LLR

* When data is packed 2 bytes/16-bit word within a data block defined by a BLK instruction, none of the data is executed until the last word of the block has been received. Thus, the nth word acts as the "execute" word in this case.

Functional Classification of Instruction Set1. Data Instructions

- | | | |
|----|---------------------|-----|
| 1. | Load & Execute Data | LXD |
| 2. | Block Transfer | BLK |

2. Control Instructions

- | | | |
|----|---------------------------|-----|
| 1. | Select Display Channel(s) | SDC |
| 2. | Select Sub-Channel(s) | SSC |
| 3. | No Operation | NOP |
| 4. | Erase | ERS |
| 5. | Interrupt | INT |
| 6. | Load Control Mode | LCM |

a. Data Modes

- | | |
|----|-------------------|
| 0. | Alphanumeric Data |
| 1. | Transverse Data |
| 2. | Raster Data |
| 3. | Complex Data |

b. Graphic Modes

- | | |
|----|-------------------|
| 4. | Graphic Vector |
| 5. | Graphic Plot |
| 6. | Graphic Cartesian |
| 7. | Graphic Element |

3. Address Instructions

- | | | |
|----|-----------------------|-----|
| 1. | Load Element Relative | LER |
| 2. | Load Element 1 | LE1 |

3.	Load Element 2	LE2
4.	Load Element Index Register	LEX
5.	Load Line Relative	LLR
6.	Load Line 1	LL1
7.	Load Line 2	LL2
8.	Load Line Index Register	LLX

2.4.1.1 BLOCK TRANSFER

The Block Transfer instruction, "BLK", provides for the packing of data, two 8-bit bytes/16-bit word, within a block containing n 16-bit words (where $0 \leq n \leq 255$).

"BLK" is provided only in conjunction with the optional 256-word, random-access Input Buffer. (Since we're dealing with packed data, the BLK instruction is meaningful only in the four data modes.)

The use of "BLK" admits of significant time and space reductions when compared with its alternative, the word-by-word output of data, one 8-bit data byte/16-bit word, provided by LXD instructions. To gain maximum benefit from both the Input Buffer and the Block Transfer Instruction, the block defined by "BLK" should be nested within a "data" string (as perceived by the user's CPU) which is output from the CPU in one very fast DMA (i.e. Direct Memory Access) burst. By letting the last word of this string be an INT instruction, the CPU is freed for other duty until the GX-100/200 processor processes the "INT" and effects an interrupt at the interface (cf. Example 3).

A block of packed data, as defined by a BLK instruction, may contain no more than 255 16-bit words (exclusive of "BLK"). If $n = 0$ "BLK" is treated as a "NOP." As soon as all n words have been received, each recognized 8-bit byte functions as though it were a full 16-bit LXD instruction. The processing of packed data proceeds from left to right at both the byte and nibble levels.

The number of nibbles in a block (Complex Data Mode) is mandatorily even.

2.4.1.1 (Ctd.)

However, the number of bytes in a block may be forced odd by setting the Termination Flag bit (i.e. making (T) = 1). This in effect causes the processor to ignore the right-most 8-bit byte of the last word in the block. Otherwise, if (T)=0, the byte count is presumed to be even, and both bytes in the last word in the block are recognized and processed as data. The one exception is given below.

- Iff
- (1) Control Mode = Transverse Data Mode, and
 - (2) Data is being output in packed format following a BLK instruction,
- then the Termination Flag, "T", is ignored by the controller thus forcing the byte count within the block of packed data to be even. All data bytes in this case are processed and displayed.

The other limitation placed on the programmer when using "BLK" is discussed below.

- Iff
- (1) System Line resolution > 256 , and
 - (2) Control Mode = Transverse Data Mode, and
 - (3) Data is being output in packed format following a BLK instruction,
- then the initial line address of the packed data must be even. (Even though (LCR)=odd, the GX-100/200 controller will ignore the least significant bit of this address -- forcing it even.)

One of the outstanding features of the BLK instruction is automatic wrap-around with line increment -- i.e. "snaking" -- after the block of packed data has been completely processed.

2.4.1.1 (Ctd.)

Wrap around on the same line occurs when the right-hand screen limit is exceeded while processing packed data just as it occurs while processing LXD instructions.

However, at the end of a block the following holds:

Assume that immediately previous to the BLK instruction $(ECR)=E$, $(LCR)=L$.

After the last word of the block has been processed ECR and LCR are revalued as follows:

$$(ECR) = E + w\Delta E, \text{ and}$$

$$(LCR) = L + h\Delta L, \text{ where}$$

$\Delta E=8$ if $(M)^*=1$, Transverse Data Mode

$\Delta E=0$ if $(M)=2, 3, \text{ or } 4$, Not " " "

$w=1$ if $(W)=0$, Single Width

$w=2$ if $(W)=1$, Double Width

$\Delta L=10$ if $(M)=0$, Alphanumeric Data Mode

$\Delta L=0$ if $(M)=1$, Transverse Data Mode

$\Delta L=1$ if $(M)=2 \text{ or } 3$, Raster or Complex Data Mode

$h=1$ if $(H)=0$, Single Height

$h=2$ if $(H)=1$, Double Height

*"(M)" = "Contents of the 3-bit Control Mode Register"

2.4.1.2 LOAD AND EXECUTE DATA

The Load & Execute Data instruction, "LXD", performs the following two functions:

- 1) loads one 8-bit data byte in the processor, and
- 2) executes on this data, causing an update of the refresh memory.

"LXD" is functional only in the four data modes.

If large amounts of data are to be output, then packing data two bytes/16-bit word after a BLK instruction is the preferred method. See 2.4.1.1.

2.4.2.1 SELECT DISPLAY CHANNEL(S)

The Select Display Channel(s) instruction, "SDC", establishes "access" to any one or more of the eight display channels provided for in GX-100/200 family. Each bit of the C-field in "SDC" pertains to one display channel. For example, iff (bit C₁)=1, is display channel 1 accessible. More than one display channel may be accessed at any one time by setting the appropriate C-bits in an SDC instruction. In fact the instruction "SDC (0-7)" allows all eight display channels to be accessed simultaneously. A zero C-field precludes ^{data output} to all display channels.

Remember that to address a particular memory channel requires setting its specific address (i. e. "cartesian") coordinates: Display Channel X, Sub-Channel Y, "SDC" provides the X, "SSC" the Y.

Refer to 2.4.2.2 for further discussion.

2.4.2.2 SELECT SUB-CHANNEL(S)

The Select Sub-channel instruction, "SSC", establishes "access" to any one or more of four sub-channels contained in a display channel. "SSC" provides the second of two required ^{channel} address coordinates. The four S-bits in "SSC" each represent one of the 4 (possible) sub-channels. Iff (bit S_i)=1, ~~for instance~~, the sub-channel 1 is ~~access~~ ^{accessed} in each of the display channels currently accessed (as defined by the last SDC instruction). Any combination of the 4 S-bits may be set. However, if no bit is set, then no data can be ^{output to the refresh memory} ~~accepted~~ until a new "SSC" is output having a non-zero S field. If all four S-bits are set, then all four sub-channels are accessed. See example below.

Example: Assume we have a 6-channel system (i.e. 6 full sets of refresh memory). Let these channels be organized (i.e., hardwired) as follows:

Display Channel 0:	4	sub-channels
Display Channel 1:	2	" "
Display Channel 2:	0	" "
:	:	
Display Channel 7:	0	" "

The six cognizable addresses are (0,3), (0,2), (0,1), (0,0), (1,3), (1,2).

The reason (1,3) and (1,2) are used rather than (1,1) and (1,0) is due to the convention calling for hardwire-addressing of sub-channels within each display channel starting with the most significant of the four bits -- 3. (note that numbering proceeds right to left: 4... 3 2 1 0.)

Now if the following two instructions . . .

2.4.2.3 NO OPERATION

The "NOP" instruction causes no action by the display system whatsoever..

It may be substituted for any instruction in a list to eliminate the execution of that instruction.

2.4.2.4

ERASE

The Erase instruction, "ERS," is a full-screen erase, and is the broadest-brush approach of 3 types of erases utilized by the GX-100/200 family. "ERS" functions identically in all eight modes -- all bits in all ^{sub-}memory channels currently accessed (as defined by the most recent ^{SSC}"SCAM" and ^{SDC}"LDC" instruction) are set equal to the contents of the 1-bit background register B (determined by the ^{status}state of the B-bit in the most recent "LCM" instruction). Being a 1-bit register, there are only two possible cases.

Case 1: (B) = ⁰∅. Clear all accessed memory bits --- erase to "black".

Case 2: (B) = ¹1. Set all accessed memory bits --- erase to "white".

2.4.2.5 INTERRUPT

The Interrupt instruction, "INT", is used only in conjunction with the 256-word random-access Input Buffer,* in the GX-100 controller and with a high-speed ^DDirect Memory Access (DMA) port in the user CPU. "INT" is to be placed as the last word of string to be output from the user CPU in one very fast burst. The CPU is then freed for use elsewhere while the GX-100 processes the instructions in the buffer sequentially. When the "INT" is ~~finally~~ processed an interrupt is generated at the interface signaling the CPU that the current supply of instruction has been depleted. The cycle is now ready to begin a new.

Other positioning and uses of INT are possible according to user needs and equipment.

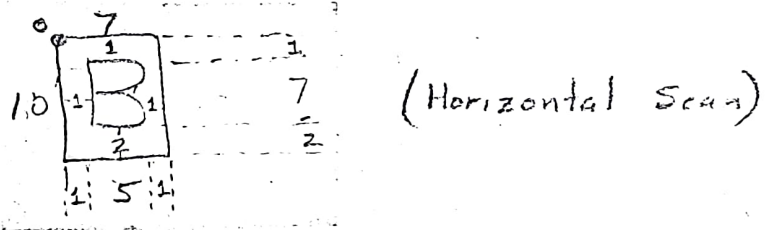
*Not available with the GX-150 processor

2.4.2.6.0

ALPHANUMERIC DATA MODE

The GX-100/200 family provides for hardware character generation of alphanumeric characters. Standard is a 64 character, 5 x 7 (within a 7 x 10 bordered matrix) ASCII font. Optional is a 96 character, 5 x 7 (within a 7 x 12 bordered matrix) ASCII font which includes lower case letters. In addition, special user-designed fonts (employing an expanded 8-bit coding scheme, if desired) will be masked by Ramtek for a nominal fee.

Characters are addressed relative to the upper left corner of the bordered matrix in a horizontal scan system (and to the bottom left corner in a vertical scan system). In other words the character origin is the upper-left (or bottom left for vertical scan) corner of the bordered matrix.



If the "H" and "W" parameters are changed, the line coordinate of the top of the bordered matrix of consecutive characters remains constant. Thus . . .



If the user prefers the above characters to be written flush on the bottom, he will have to output appropriate LL1 (or LLR) instructions between ASCII data and the LCM instruction required to effect the above indicated sizing changes involving "H" and "W". (cf. example below, mnemonics only.)

2.4.2.6.0 (CTD)

AB ⊆ DEF

LCM, AN

LXD, A

LLR, -8

ie. raise ^{the} main

LCM, AN, H,

LXD, B

LLR, +8

ie. lower the main

LCM, AN, W

LXD, C

LLR, -8

LCM, AN, H, W

LXD, D

LLR, +8

LCM, AN

LXD, E

LXD, (F)

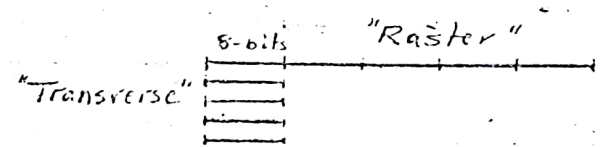
Note that software character generation is normally most easily accomplished using Transverse Data Mode. (cf. 2.4.2.6.1)

2.4.2.6.1 TRANSVERSE DATA MODE

Transverse Data Mode provides the user with a technique for "painting" data on the screen in 8-element wide swathes running perpendicular to the raster lines. It is particularly useful for describing any special alphanumeric characters and/or graphic symbols required by the user. Cyrillic fonts, Arabic fonts, etc., meteorological, chemical, astronomical (even astrological) symbols, etc. -- all these and more are very neatly handled using this mode.

Transverse Data Mode is in many ways similar to Raster Data Mode. Both involve the bit-by-bit ^{mapping} "translation" (as influenced by the background bit in "LDM") of an 8-bit data byte into 8 consecutive screen elements along a raster line. Whereas in Raster Data Mode ensuing data bytes are written "serially" down the initial raster line (i.e., 8-element long line segments laid end-to-end), in Transverse Data Mode ensuing data bytes are written "in parallel," each starting at the same initially element address, but on the next raster line, resulting in a pattern of 8-bit long line segments oriented like the rungs of a ladder (Figure 2.4.3).

Figure 2.4.3



When using "BLK" in Transverse Data Mode special care must be taken to start each block on an even-numbered line address. Also, only an even number of bytes may be output. See 2.4.1.2.

2.4.2.6.3 COMPLEX DATA MODE

Complex Data Mode involves a four-to-one mapping of (Z-axis) data to single screen elements^(pixels). Each data byte contains two nibbles, each nibble providing Z-axis information for 1 screen element. Consecutive nibbles describe consecutive screen elements ~~serially~~ in a manner similar to Raster Data Mode.

The Z-axis information (i.e. one 4-bit nibble) acts in a manner analogous to the 4 bit access mask employed in SSC instructions. The principal difference between the two lies in the fact that the access mask remains set until reset by another "SSC", where Z-axis (i.e. "intensity") data is volatile, being employed just once to describe just one element. Z-axis nibbles in no way affect the access mask. (cf. Example 3).

Complex Data Mode is very useful in applications in which intensity data is important (such as all types of spectro-analysis), in pictorial graphics (where gray-scaling and color are important), and in any situation calling for point-by-point description of Z-axis information.

2.4.2.6.2 RASTER DATA MODE

Raster Data Modes provides a bit/element/channel *bit*-image description of a display. Each 8-bit data byte (from an "LXD" or following a BLK instruction) describes the status of eight consecutive elements (as mediated by the background bit, cf. 2.4.4.2) along one raster line in all currently-accessed channels. The next data byte which is output describes the next eight consecutive elements along the same scan line.

Mapping from data bit to screen element (1) is one-to-one, (2) is affected by the background bit, and (3) proceeds from left-to-right in the data byte and from the beginning to the end of the scan line.

When one comes to the end of a raster line, one of two situations prevails:

- 1) If data is being output within a block of packed data as described by "BLK", then new data is written starting at the beginning of the next raster scan line; *iff the last pixel was described by the otherwise last word of the block, otherwise wrap-around on the same line occurs.*
- 2) If data is being output via an LXD instruction, it is recorded at the beginning of the same raster line unless the next raster line is specifically designated by an LL1 or LLR instruction intercalated between the previous LXD instruction and the one in question.

In Raster Data Mode, data bytes are written "serially" along the same raster line, while in Transverse Data Mode the bytes are written "in parallel" on consecutive scan lines, starting at the same element address each time.

Cf. Figure 2.4.3 on previous page.

2.4.2.6.4 GRAPHIC VECTOR MODE

Graphic Vector Mode provides a convenient method for hardware generation of vectors by specifying the two end-points necessary to describe a vector. Thus an instruction sequence "LL1", LE1", LE2/LER", and "LL2/LLR" would cause a vector to be drawn. If one wished to draw a sequence of connected vectors, one would continue to output "LE2/LER" and "LL2/LLR" instructions.

(Note that either "LL2" or "LLR" commences execution of the vector in a horizontal-scan system and thus must be the last instruction of the sequence of instructions defining the vector. cf. Example 4.)

2.4.2.6.5 GRAPHIC PLOT MODE

Graphic Plot Mode is the mode most often used to plot curves (filled or unfilled). When viewed "microscopically" a "curve" in this sense actually consists of a sequence of ^{overlapping} vertically-overlapped vertical line segments as shown below.



When seen on the screen, the curve appears continuous and smooth(er).

Case 1: In a horizontal-scan system after initializing the address (by issuing "LE1/LER" and "LL1" commands) each and every "LL2" or "LLR" causes one line segment to be drawn at the current element address, (ECR), from the last-specified (or current) line address, (LCR), to the new line address specified by "LL2" or "LLR". The current element address is then incremented by one -- i.e., $(ECR) \leftarrow (ECR) + 1$. Also, (LCR) is revalued in accordance with principles discussed in 2.4.4.5.

Case 2: In vertical systems, the line segments are again vertical, but the functioning of LL1 and LE1, LE2 and LER instructions are reversed -- "LE2" or "LER" become the vehicles for outputting new address data.

"Solid" or "filled-in" curves are made possible by setting the Fixed-point Graphics Flag, "F", in an LCM instruction. In this case, one end of each

vertical line segment is set to the value of the "y" index register -- i.e. "(LXR)" in a horizontal system, "(EXR)" in a vertical system. (Remember that if (F)=0, one end of each vertical line segment = (LCR) (= (ECR) in the vertical scan system).

delete ~~Note that Graphic Element Mode is a special case of Graphic Plot Mode.~~

~~Here points rather than line segments are the primitives of the curve.~~

2.4.2.6.6 GRAPHIC CARTESIAN MODE

Graphic Cartesian Mode provides for drawing solid rectangles. When reverse background is in effect, GCM becomes especially useful as an area erase ^{of} display material of unlimited complexity within any rectangularly-bounded area.

Two element coordinates and two line coordinates are necessary to describe a rectangle. The Element and Line Current-value Registers provide two of the necessary coordinates. The last two must be furnished by LE2/LER and LL2/LLR instructions. The order in which these two instructions are issued is critical and is discussed more thoroughly in 2.2.1. Note also that issuing LE1 and/or LL1 instructions causes ECR and/or LCR (above) to be revalued. In many instances, therefore, the programmer will issue four instructions to draw a rectangle:

LE1, LL1, ~~LE1~~₂/LER, LL2/LLR. However, this is not mandatory -- LE2/LER and LL2/LLR will suffice if (ECR) and (LCR) are correct already.

2.4.2.6.7 GRAPHIC ELEMENT MODE

Graphic Element Mode enables the programmer to describe single screen elements. For each such element, two instructions are required -- LEI/LEER and LL2/LLR (assuming horizontal scan).

Note: Unlike Graphic Plot Mode, there is no automatic increment of LCR. Both LCR & ECR contain the current screen coordinates of the latest element described.

* as influenced by the Address-Addressing Page

2.4.3.0 ADDRESSING

The eight address instructions may be divided into four categories:

- 1) "L α R" =: Load α Relative
- 2) "L α 1" =: Load α 1
- 3) "L α 2" =: Load α 2
- 4) "L α X" =: Load α Index Register

where $\alpha \in \{E, L\}$ (i.e., " α " is either "E" or "L").

There are three methods of addressing:

- 1) Direct: i.e. "L α 1" or "L α 2" with (I) = 0
- 2) Indexed: i.e. "L α 1" or "L α 2" with (I) = 1
- 3) Relative: i.e. "L α R"

The 10 address bits within each address instruction function in one of two ways:

- 1) as an absolute screen address -- i.e. "Direct Addressing"
- 2) as a two's - complement displacement to be summed with the contents of one of the two "bias" registers
 - a) Index Register (i.e. EXR or LXR) -- i.e. "Indexed Addressing,"
 - or,
 - b) Current-value Register (i.e. ECR or LCR) -- i.e. "Relative Addressing".

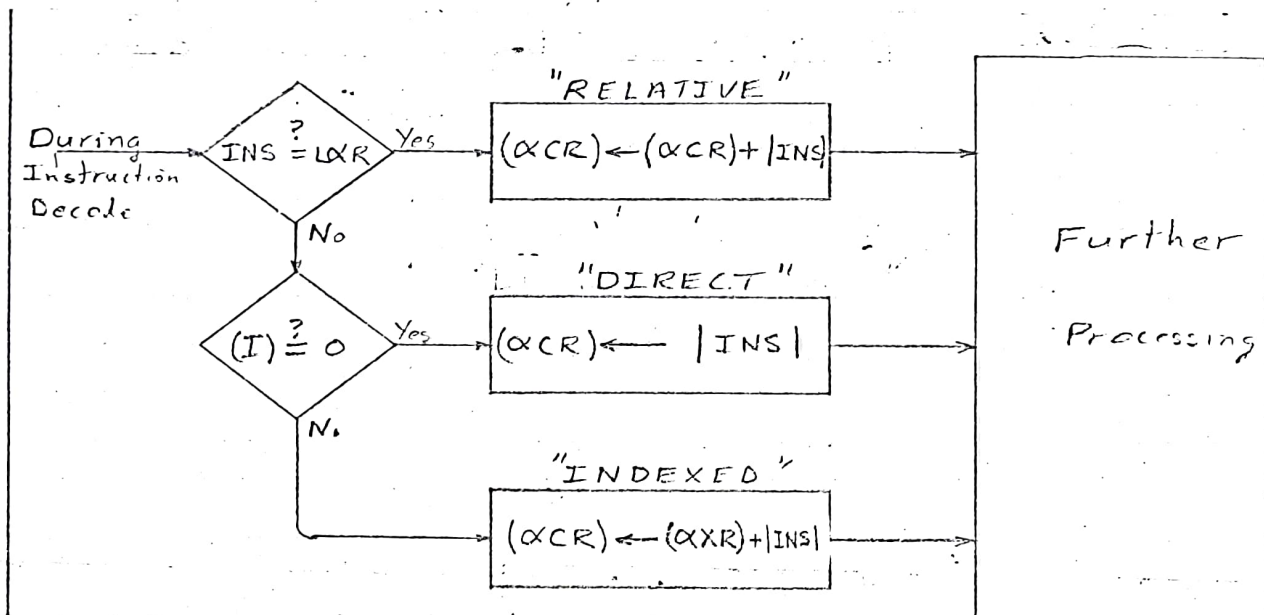
In either case what results is an absolute screen address which is then loaded into the α Current-value Register (i.e. ECR or LCR). It is important to realize that all eight internal address registers (cf. 2.1.2) at all times contain absolute screen addresses.

2.4.3.0 (CTD)

Each and every LE1, LE2, and LER instruction received and decoded causes the Element Current-value Register to be revalued. Each and every LL1, LL2, and LLR instruction similarly causes the Line Current-value Register to be revalued. Revaluation of ECR and/or LCR to a new absolute screen address involves the following three factors:

- 1) the type of instruction: written generically as "INS";
- 2) the indicated address (10-address bits): written $|INS|$; and
- 3) the status of Indexed Addressing Flag, "I".

The above considerations are summarized in the following flow chart:



where $\alpha \in \{E, L\}$ (i.e. " α " is either "E" or "L"), and " $|INS|$ " = 10 address data bits obtained from the address instruction "INS" being processed.

2.4.3.1 LOAD ELEMENT RELATIVE

A Load Element Relative instruction, "LER", may serve two distinct functions:

- 1) In all cases the 10 data bits of the instruction are added (using 2's complement arithmetic) to the contents of the Element Current-value Register. This sum is then loaded into ECR to become the new current (absolute) element address. $(ECR) \leftarrow (ECR) + \{LER\}$
- 2) If the system employs a vertical scan, then in the four graphic modes once the above function has been completed, execution of the pending graphic operation commences. That is, a vector, rectangle, or "point" is drawn on the screen, depending on the mode specified.

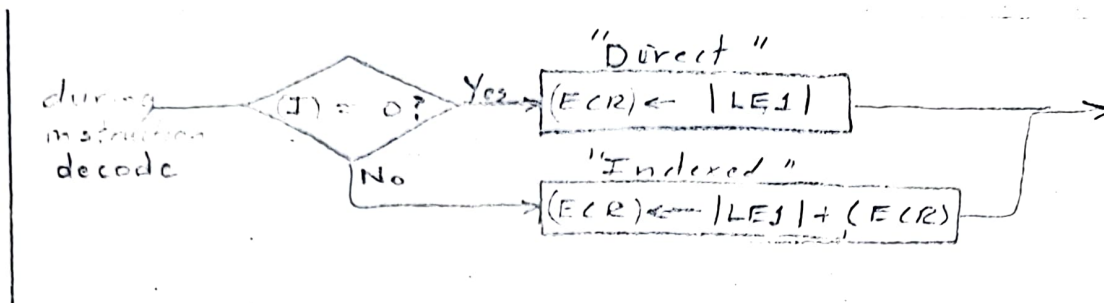
2.4.3.2 LOAD ELEMENT 1

The Load Element 1 instruction, "LE1", provides the first of two element address for the four graphic modes, while providing (as may "LER") the one and only element address required for the four data modes. When (I) = 0, the indicated element address, $\{LE1\}$, is treated as a 2's-complement displacement to be added to the contents of the Element Index Register, (EXR), to provide the effective current absolute element address which is then loaded into the Element Current-value Register, ECR.

Under no circumstances does "LE1" function as an "Execute" instruction (cf. Table 4.1).

In Graphic Vector and Graphic Plot Modes, when (F) = 1, "LE1" is neither required nor (graphically) functional. It will, however, cause ECR to be revalued as shown below.

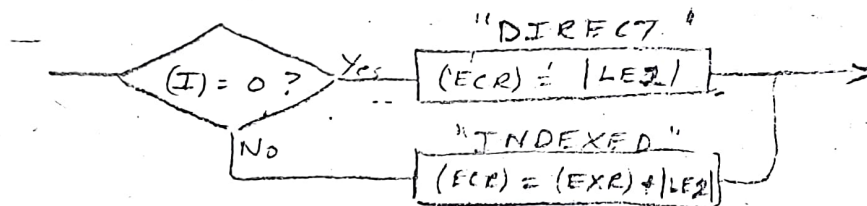
The following flow chart is valid in all eight control modes.



2.4.3.3 LOAD ELEMENT 2

A Load Element 2 instruction, "LE2", may serve two distinct functions:

- 1) In all occurrences it provides a new current element address, (ECR), as charted below.



- 2) Iff the system employs a vertical scan, then in the four graphic modes, once the above function has been completed, execution of the pending graphic operation commences. That is, a vector, rectangle, or "point" is drawn on the screen, depending on the mode specified. ("LER" instruction will have the same affect.)

2.4.3.4 LOAD ELEMENT INDEX REGISTER

The Load Element Index Register instruction, "LEX", causes the Element Index Register to be loaded directly with $|LEX|$. (i.e. the 10 address data bits within EXR).

$$(EXR) \leftarrow (LEX)$$

(EXR) is used when (I) = 1 as a "bias" register during the processing of LE1 and LE2 instructions. When in Graphic Vector or Graphic Plot Mode, iff (F) = 1, (EXR) replaces (LE1) as the first of two required element addresses.

2.4.3.5 LOAD LINE RELATIVE

A Load Line Relative instruction, "LLR", may serve two distinct functions:

- 1) In all cases the 10 data bits of the instruction are added (using 2's - complement arithmetic) to the contents of the Line Current-value Register. This sum is then loaded into LCR to become the new current (absolute) line address.

$$(LCR) \leftarrow (LCR) + (LLR)$$
- 2) Iff the system employs a horizontal scan, then in the four graphic modes once the above function has been completed, execution of the pending graphic operation commences. That is, a vector, rectangle or "point" is drawn on the screen, depending on the control mode specified.

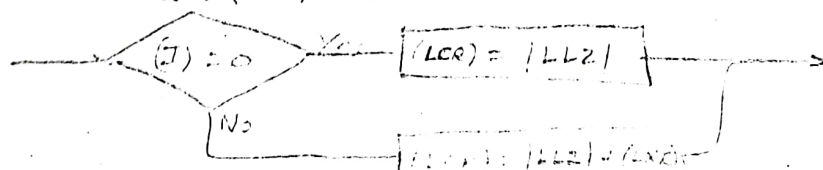
2.4.3.6 LOAD LINE 1

Functionally equivalent to "LE1", cf. 2.4.3.2

2.4.3.7 LOAD LINE 2

A Load Line 2 instruction, "LL2", may serve two distinct functions:

- 1) In all occurrences "LL2" provides a new current line address - i.e. (LCR) -- as charted below.



where $\{LL2\}$ =: the 10 address data bits of "LL2"

- 2) Iff the system employs a horizontal scan, then in the four graphic modes, once the above function has been completed, execution of the pending graphic operation commences. That is, a vector, rectangle, or "point" is drawn on the screen, depending on the mode specified. (An LLR instruction has the same effect.)

2.4.3.8 LOAD LINE INDEX REGISTER

The Load Line Index Register instruction, "LLX", causes the Line Index Register to be loaded directly with (LLX) (i.e. the 10 address data bits within "LXR").

$$(LXR) \leftarrow \{LLX\}$$

(LXR) is used when (I)=1 as a "bias" register during the processing of LL1 and LL2 instructions. When in Graphic Vector or Graphic Plot Mode, iff (F) = 1, (LXR) replaces $\{LL1\}$ as the first of two required line addresses.

where $\{LL2\} =$ the 10 address data bits of "LL2"

- 2) Iff the system employs a horizontal scan, then in the four graphic modes, once the above function has been completed, execution of the pending graphic operation commences. That is, a vector, rectangle, or "point" is drawn on the screen, depending on the mode specified. (An LLR instruction has the same effect.)

2.4.3.8 LOAD LINE INDEX REGISTER

The Load Line Index Register instruction, "LLX", causes the Line Index Register to be loaded directly with $\{LLX\}$ (i.e. the 10 address data bits within "LXR").

$$(LXR) \leftarrow \{LLX\}$$

(LXR) is used when (I)=1 as a "bias" register during the processing of LL1 and LL2 instructions. When in Graphic Vector or Graphic Plot Mode, iff (F) = 1, (LXR) replaces $\{LL1\}$ as the first of two required line addresses.

2.4.4.1 ADDITIVE-WRITE FLAG

The additive-write Flag, A, is a control parameter used in only three modes --
Alphanumeric Data, Transverse Data and Raster Data Modes.

If (A)=0, write "1's and 0's" -- write with Replacement,

If (A)=1, write "1's only" -- write Additively.

The above summary will now be expanded below.

Only in two modes (Transverse Data Mode and Raster Data Mode) is "core-image" data available immediately without scan-conversion. In the other five modes data is generated by the (Ramtek)processor from a sequence of one or more instructions. The "raw"^{scan}data" derived either directly from an "LXD" or indirectly from a hardware generator (such as the alphanumeric character generator) consist either of 1's only or 1's and 0's. Only in the three modes listed above are 1's and 0's generated. Only here, then, is the Additive-write Flag applicable.

Regardless of how this raw scan data has been derived, it must be "filtered" before being processed further. This is done as follows:

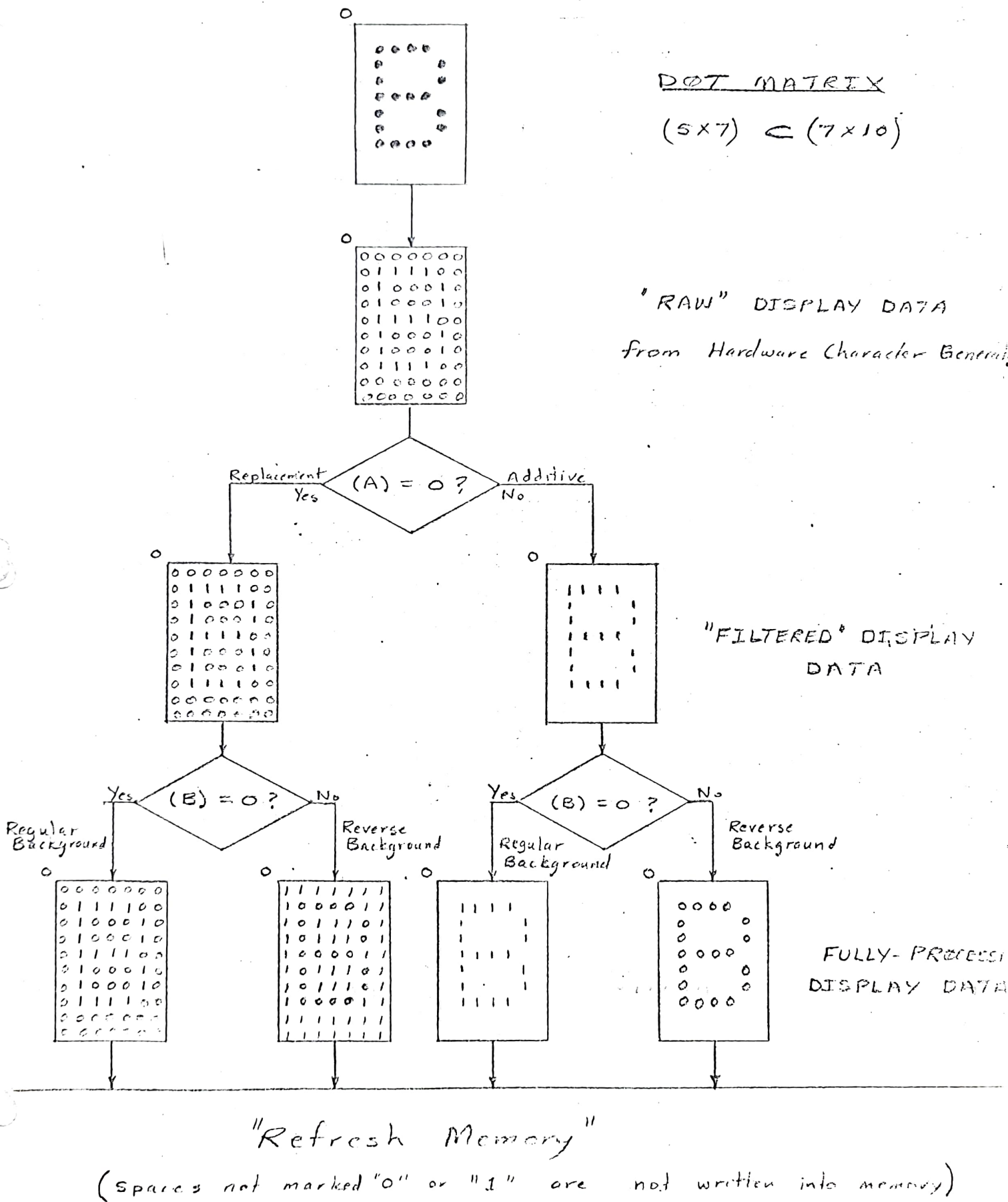
(A)=0: All raw data, both 1's and 0's, are maintained.

(A)=1: Only the 1's in the raw data are recognized and maintained.

The 0's are at this time discarded.

Raw display data has now become filtered display data. This filtered display data is then logically "exclusively-or'd" bit-by-bit with the background bit B. It is the logical product of this operation which is at last written into the refresh memory to be displayed on the screen.

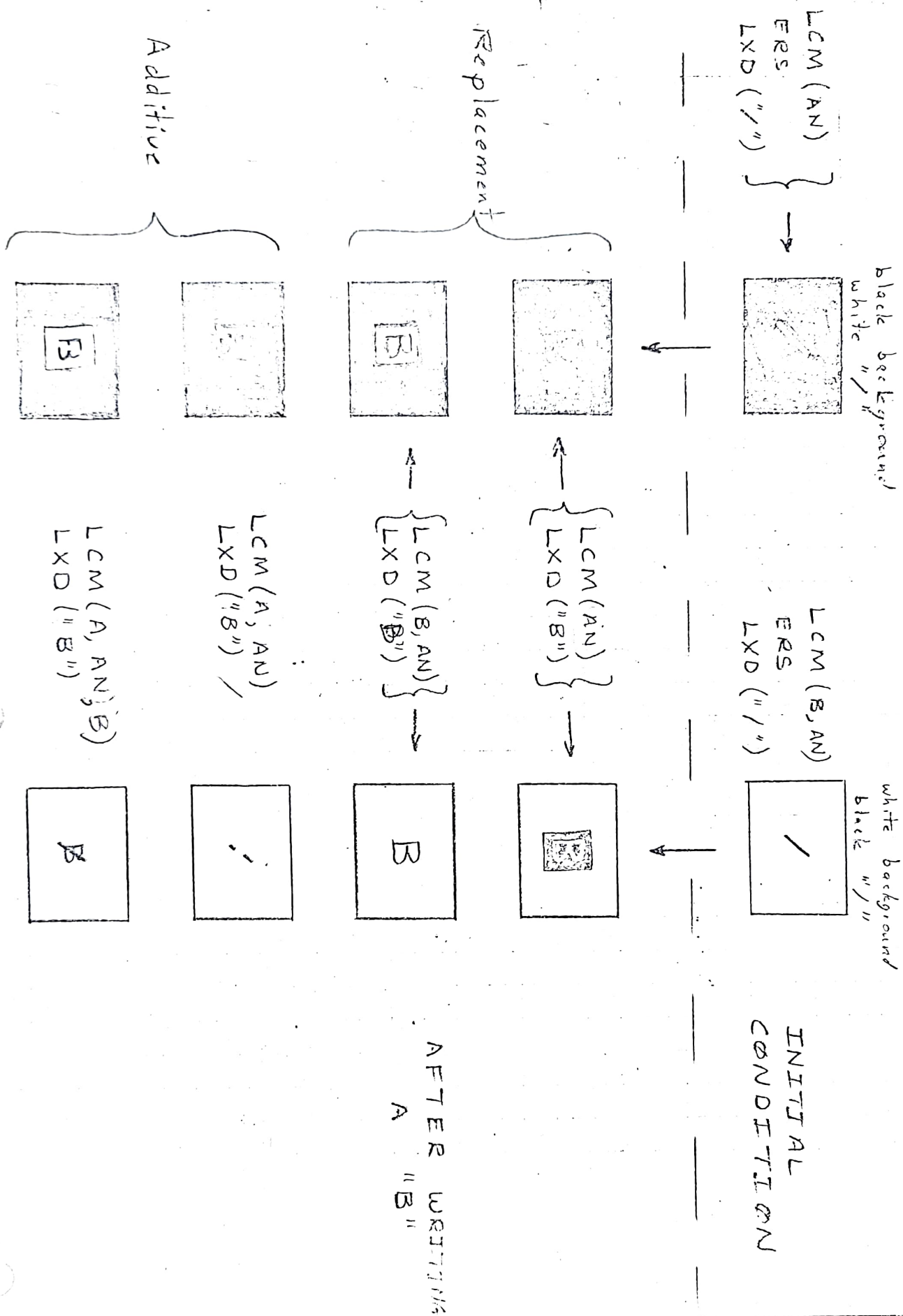
FIGURE 2.10



Use of (A) & (B)

FIGURE 2.11

JME



2.4.4.2 BACKGROUND SELECT FLAG

Each time an LCM instruction is received and decoded, a 1-bit "B" register is loaded with the Background Select Flag contained in "LCM". The contents of this register remain constant until another LCM instruction (having the Background Select Flag set to the opposite polarity) is decoded.

The status of "B" determines the white-on-black vs. black-on-white relationship. Logically, what happens is that each bit of ^{raw scan} data generated by the processor (from instructions fed to it by the user's CPU) is "Exclusively-or'd" with the contents of the background select register. The logical product is then written into the refresh memory. In symbolic form this becomes. . .

$$(\text{data bit}_i) \vee (B) \rightarrow (\text{refresh memory bit}_i)$$

. . . where "()" is read "the contents of."

During a screen erase (i.e. "ERS"), $(\text{data bit}_i) = 0$ for every "i", and the above relationship holds. Put another way, an ERS instruction sets all bits in all accessed memory channels = (B).

Thus, if $(B)=0$, erase to "black" background, but if $(B)=1$, erase to "white" background.

Figure 2.11 illustrates all the above points in detail.

2.4.4.3 FIXED-POINT GRAPHICS FLAG

The Fixed-point Graphics Flag, F, is used only in Graphic Vector Mode and Graphic Plot Mode. The status of the Fixed-point Graphics Flag is used to load a 1-bit register (also called "F"). The contents of "F" remain constant until changed by an "LCM" (Vector or Plot Mode) with "F" set to different polarity. The content of "F" is then used to determine whether the first of two sets of absolute address coordinates is to be obtained from the Element and Line Current-value Registers or from the Element and Line Index Registers. That is,

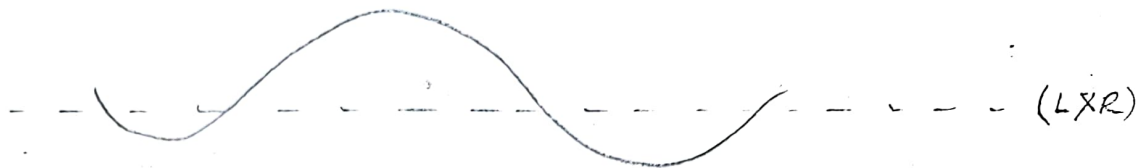
if (F)=0, $E1=(ECR) \ \& \ L1=(LCR)$, while if (F)=1, $E1=(EXR) \ \& \ L1=(LXR)$,

where E1 and L1 taken together form the first of two required sets of cartesian coordinates. (The second set is derived from LE2/LER and LL2/LLR instructions.) Practically, what this means is that if (F)=1, LE1 and LL1 instructions will have no direct effect on the vector or graph being drawn. Their function is taken over by the two index registers. (Note though, that the ECR and LCR will be revalued by each LE1 and LE2 instruction issued. Thus if the next instruction is an "LER" or "LLR", which involves summing a two's complement displacement with (ECR) or (LCR), the final positioning of that second set of coordinates may be drastically affected.)

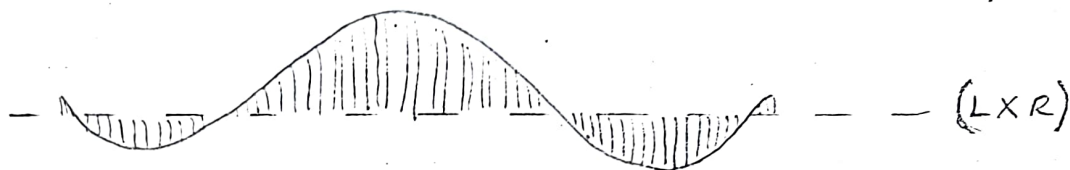
The major effect of the case (F)=1 is to allow plotted curves to be "filled-in" (upwards or downwards as the case may be) to a constant y-coordinate given as (LXR), and to allow vectors to be drawn, each emanating from the coordinate (EXR), (LXR). Thus polar coordinate^s, radar sweeps, etc., can be rather easily handled. The above examples are illustrated in Figure 2.14.

Graphic Plot Mode:

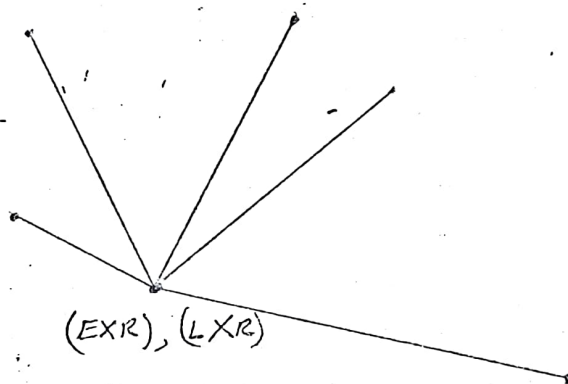
$(F) = 0$



$(F) = 1$

Graphic Vector Mode:

$(F) = 1$



2.4.4.4 DOUBLE-HEIGHT FLAG

The Double-height Flag, "H", determines whether display data generated in any of the four data modes is to be written single or double height. That is,

if (H)=0, write single height, but if (H)=1, write double height.

2.4.4.5 INDEXED-ADDRESSING FLAG [insert from next page]

2.4.4.6 DOUBLE-WIDTH FLAG

The Double-width Flag, "W", determines whether display data generated in any of the four data modes is to be written single or double width.. That is,

if (W)=0, write single width, but if (W)=1, write double width.

2.4.4.7 TERMINATION FLAG

The Termination Flag, "T", of a BLK instruction determines which of the 2 bytes in the last 16-bit word of a block of packed data terminates that block's significant data. That is,

if (T)=0, process both bytes, but if (T)=1, process only the left-most byte, and ignore the right-most byte.

"T" thus allows the programmer to indicate whether the byte count is to be even or odd within the block.

Note: "T" is inoperative in Transverse Data Mode.
Here the byte count is always assumed to be even.

2.4.4.5 INDEXED-ADDRESSING FLAG

The Indexed-addressing Flag, "I", present in every LCM instruction, causes a 1-bit register (also called "I") to be set. This register mediates between "absolute" and "indexed" addressing in LE1, LE2, LL1, and LL²~~1~~ instructions such that

(I)=0 $\Rightarrow$ absolute addressing ,

(I)=1 $\Rightarrow$ indexed addressing

where indexed addressing uses the 10-bit address data of the address instruction as a two's-complement displacement to be summed with the contents of the index register, the sum being placed in the current-value register as the current (absolute) screen address. ("I" does not apply to LER & LLR instructions.)

EXAMPLE 0: USE OF ALPHA-NUMERIC DATA MODE

- a) Task: Given a three-channel color system, write the word "Rôle" at (4, 5) in red against a blue-green background. As per convention assign ^{sub-channel} Ch 3 to red, ^{Sub-channel} Ch 2 to green, ^{Sub-channel} Ch 1 to blue, with Ch 0 being unused. Assume we're using Display Channel 0.

b) Procedure

- 1) First output the following block of instructions from user CPU to GX-100/200 to initialize system

HEX	OCTAL	MNEMONIC	DESCRIPTION
0801	004001	SDC, 0	Access Display Channel 0
1C00	016000	LCM, AN	Regular background
0C08	006010	SSC, 3)	
1400	012000	ERS	Erase the red channel to a "black" background.
1D00	016400	LCM, AN, B	Reverse background
0806	006006	SSC, 1, 2	
1400	012000	ERS	Erase green & blue channels to "white" background -- i.e., screen should now be completely bluegreen.
2404	022004	LE1, 4	Set element address = 4
3405	032005	LL1, 5	Set line address = 5
1D80	016600	LCM, AN, A, B	Ready system to write additively with reverse background. Initialization now complete.

EXAMPLE 0: (CTD)

b) Procedure (CTD)

- 2) Next write "Rôle" in "black" (by selective erasure of the letters from the blue-green background). This is done by outputting the following data:

HEX	OCTAL	MNEMONICS	DESCRIPTION
0052	000122 ✓	LXD, R	<i>mod 400</i>
005E	000136	LXD, (^)	
2379	021771 <i>020371 for 3200 resolution</i>	LER, -7 <i>Not</i>	Equivalent to a backspace.
006F	000157	LXD, C <i>000122</i>	<i>20433</i>
006C	000154	LXD, 1 <i>122</i>	
0065	000145	LXD, (e) <i>122</i>	

- 3) The last step is to restore normal background and ^{to} write the above data in red, additively -- i.e., so that when we backspace (via "LER, -7") and write the "0" it won't obliterate the circumflex, (^). This is done by outputting the following data from the CPU:

HEX	OCTAL	MNEMONICS	DESCRIPTION
1C80	016200	LCM, AN, A	Set system to write additively (for the "0"). with normal background.
0C08	006010	SSC, 3	Access the red channel
2404	<i>157</i> 022004	LE1, 4 <i>50</i>	Re-address to initial
3405	<i>110</i> 032005	LL1, 5 <i>50</i>	coordinates

HEX	OCTAL	MNEOMICS	DESCRIPTION
0052	000122	LXD,R	
005E	000136	LXD,^	
23F9	021771 52-477	LER,-7	
006F	000157	LXD,6	
006C	000154	LXD,1	
0065	000145	LXD,e	End.

4 0 31

EXAMPLE 1: Use of Transverse Data Mode,

"Indexed" addressing, & Block Transfer.

a) Task: In user's core is located a table of special characters, out of which one character, the Greek letter gamma, γ , is to be displayed on the screen at $(\overset{15}{\cancel{11}}, \overset{11}{\cancel{75}})$ on sub-channels 1 and 2 of display channel 1--ie, on $\langle 1, 1 \rangle$ and $\langle 1, 2 \rangle$

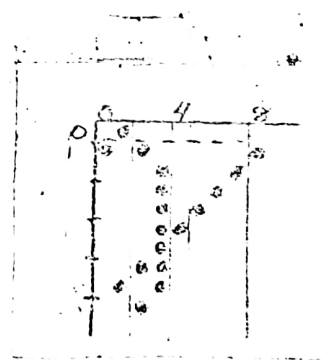
b) Procedure:

Output the following string of words:

HEX.	OCTAL	MNEMONIC	DESCRIPTION
0802	004002	SDC, 1	Select display channel
0C06	006006	SSC, 1, 2	Select sub-channels 1 & 2
1E01	017001	LCM, TD, I	Transverse Data Mode with indexed addressing
2C0F	026017	LEX, 15	Set (EXR) = 15
3C0A	036012	LLX, 10	Set (LXR) = 10
2400	002000	LEI, 0	$(ECR) = (EXR) + (LEI)$ $= 15$
3400	022000	LL1, 0	$(LCR) = (LXR) + (LL1)$ $= 10$

EXAMPLE 1: (CTD)

HEX	OCTAL	MNEMONIC	DESCRIPTION
0040	000100	LXD, X'40'	Describe 1st point of the gamma. (LCR)=1 Now we may use 'EL' because (LCR) is even
0405	020005	BLK, 5	Prepare to output 10 ms of packed data. Even though we only really need 11 bytes, the last byte must be set = 0 because the Terminate Flag is inoperative in Transverse Data.
A011	120021	DTA	
1214	011024 012021	DTA	
1810	014020	DTA	
3050	030120	DTA	
<u>2000</u>	<u>020000</u>	DTA	now (ECR)=23 (LCR)=1
0080	000200	LXD, X'80'	last point of gamma



EXAMPLE 2: Use of Raster Data Mode

a) Task: Starting at (20, 40) draw a dashed line to (51, 40) such that the "white" line segments are 4 elements long and the "black" line segments are 5 elements long. Use display channel 2, sub-channel 3, i.e., <2, 3>.

b) Procedure: Set up the following data in CPU core then output it to GX-100/200.

HEX.	OCTAL	MNEMONICS	DESCRIPTION
0402	004002	¹² SDC, 2	Select channel
0C08	006010	SSC, 3	<2, 3>
1C02	016002	LCM, RD LLI ^{40_{oct}}	Select Raster Data Mode
00F0	000360	LEI ^{25_{oct}} LXD, X'F0'	Load necessary data
0078	000170	LXD, X'78'	
003C	000074	LXD, X'3C'	
001E	000036	LXD, X'1E'	

Example 3: Use of Complex Data Mode, High-speed Input Buffer and
BLK instruction.

Task: Given a system resolution of 256 lines x 320 elements (horizontal scan), construct a voice spectrograph such that X=time, Y=frequency, and Z=amplitude (where Z is 4 bits deep -- i.e., 16 levels gray-scale, or color) in display channel 2. Amplitude data we will assume exists in mass store and is assembled in four 4-bit nibbles/16-bit word preparatory to being output to the GX-100/200. We'll also assume that the user CPU is equipped with a DMA channel (or its equivalent) to allow very fast output of ≤ 256 ~~(consecutive)~~ 16-bit words per "burst".

Procedure: Set up the following 250-word data file in core. Output the entire file in one continuous burst to GX-100/200. Leave the external interrupt system enabled so that when the GX-100/200 has processed the last word of the high-speed Input Buffer, this last instruction, "INT", will effect an interrupt. The Input Buffer is then refilled -- the process continuing until Z-axis data on all 256 scan lines have been described.

HEX	OCTAL	MNEMONICS	DESCRIPTION
0802	004002	SDC,2	Select display channel 2.
0C0F	006017	SSC,3,2,1,0	Erase all 4
1C03	016003	LCM,CD	sub-channels within
1400	012000	ERS	this group.

EXAMPLE 3: (Ctd.)

HEX	OCTAL	MNEMONICS	DESCRIPTION
2400	022000	LE1, 0	Address 1st element of
3400	032000	LL1, 0	1st raster line ,
0450	020120	BLK, 80 _{dec}	80 packed data words follow
....		DTA	describing 320 pixals.
....		.	Only after last word of block has been
....		.	received does <u>execution</u> on the data occur
0450	020120	BLK, 80	(ECR)=0, (LCR)=1 Output 80
....		DTA	packed data words.
....		.	Total of 320 Nibbles each describing
....		.	1 pixal with 4-bit intensity info.
0450	020120	BLK, 80	(ECR)=0, (LCR)=2
....		DTA	
....		.	
....		.	
1800	014000	INT	Last word of file. Effects interrupt
			of host CPU. (ECR)=0, (LCR)=3

Further files need contain only "BLK", "DTA" and "INT" instructions. No readdressing is necessary.

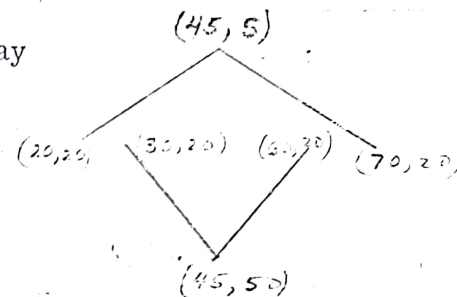
Example 4: Use of Graphic Vector Mode and Fixed-point Graphics

Task: Given a 12-channel system arranged into 3 display

channels each consisting of 4 sub-channels, con-

struct the accompanying figure in sub-channels

0 & 3 of display channels 1 & 3.



Procedure: Output the following data:

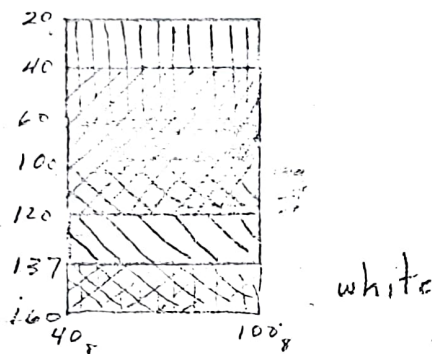
HEX	OCTAL	MNEMONIC	DESCRIPTION
040A	004012	SDC,1,3	Access the 4
0C09	006011	SSC,0,3	sub-channels in question
1C0C	016014	LCM,GV,F	Graphic Vector Mode Fixed-pt. Graphics.
2C2D	026055	LEX,45	
3C32	036062	LLX,50	
281E	024036	LE2,30	
3814	034024	LL2,20	(45, 50), (30, 20)
283C	024074	LE2,60	
3814	034024	LL2,20	(45, 50), (60, 20)
1C04	016004	LCM,GV	regular vector graphics
2414	022024	LE1,20	
3414	032024	LL1,20	
282D	024055	LE2,15	
3805	034005	LL2,5	(20, 20), (45, 5)

HEX	OCTAL	MNEMONIC	DESCRIPTION
2846	024070	LE2,70	
300F	034017	LLR,15 (or LL2,20)	(45,5), (70,20)

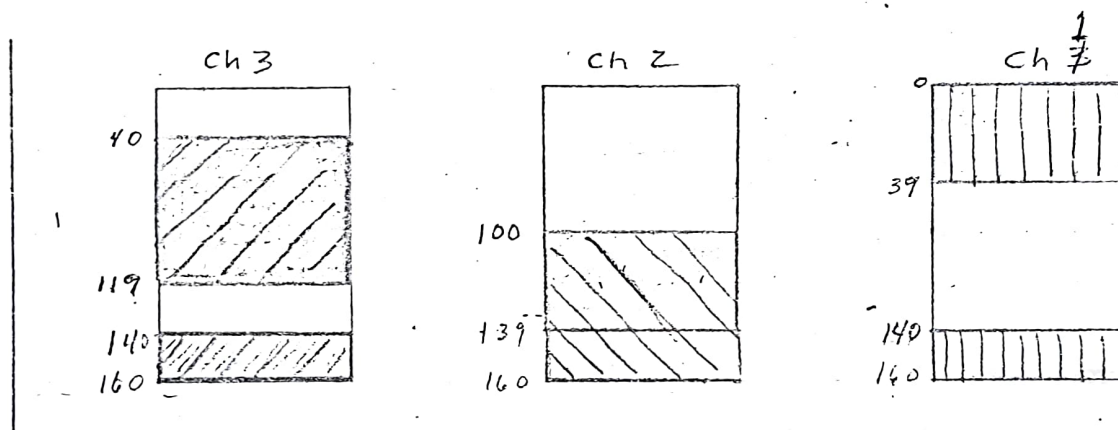
EXAMPLE 6: Use of Graphic Cartesian Mode, and Relative Addressing.

Task: Assume we have a 3-channel color system, 256 (lines) x 320 (channel) ^{elements}.

(By convention, Ch 3= red, Ch 2 = green, Ch 1 = blue, Ch 0 not used.) Construct a picture as shown in figure ____.



When each of the 3 channels are looked at separately the following patterns are evidenced:



Example 6:

HEX	OCTAL	MNEMONIC	DESCRIPTION
		<i>[rewrite comma for parentheses]</i>	
1606	016006	LCM (GC)	
0C08	006010	SSC (3)	Red rectangle
2428	022050	LE1 (40)	
2864	024144	LE2 (100) or LER (60)	
3428	032050	LL1 (40)	
304F	030117	LLR (79) or LL2(119)	
1000	010000	- box is now drawn NOP	
0C04	006004	SSC (2)	Green rectangle
2828	024050	LE2 (40)	
348B	032213	LL1 (139)	
33D9	031731	LLR (-39)	
0C02	006002	SSC (1)	Blue rectangle
203C	020074	LER (60)	
3427	032047	LL1 (39)	
3800	034000	LL2 (0)	
1000	010000	NOP	
0C0E	006016	SSC (3,2,1)	White rectangle
23C4	031704	LER (-60)	
348C	032214	LL1 (140)	
3014	030024	LLR(20)	

2.6.0 ERASURE

The GX-100/200 family provides for three distinct methods of erasing display data.

These are as follows:

1. Full-screen erase using a single "ERS" instruction.
2. Area erase using Graphic Cartesian Mode with reverse background.
3. Character retrace erase whereby the block of data used to construct the chosen screen characters is output a second time, only this time with the background reversed from the previous setting. (Not applicable to Alpha-numeric Mode).

The first two methods are normally employed much more frequently than is the third.

3.0.1. Full-screen erase is discussed separately as the "ERASE" instruction

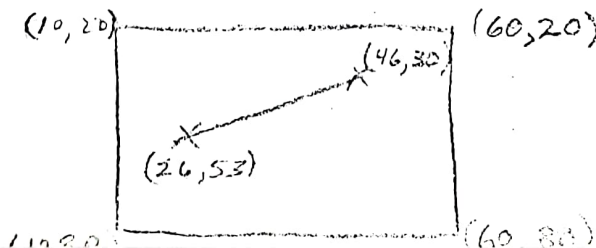
(cf. 2.1.4)

3.0.2 Area erase is capable of erasing data of any complexity, occurring within

← a rectangular area, and written in any combination of channels within a display group.

By making the rectangle as big as the system resolution (e.g. 256 x 320), a full-screen erase, equivalent to that obtained from a single "ERS" instruction, may be obtained. Normally, however, only a small portion of the screen is to be erased, and the Graphic Cartesian Area Erase is the method of choice.

Example: Assume that within the bounds of a rectangle described by the points (10,20), (60, 20), (60, 80), (10, 80) is drawn a vector in red (i.e. Ch 3) which is labeled in green with alpha-numeric characters as shown in figure _____. (The rectangle itself is not actually drawn on the screen -- rather, it is an arbitrary construct of our own devising -- one of a number of such rectangles that could have been chosen to enclose the desired area of erasure.)



For our particular example the block of data necessary to effect the erasure is as follows:

3-0-2

HEX	OCTAL	MNEMONIC	DESCRIPTION
1D06	016406	LCM(B, GC)	Graphic Cartesian Mode with reverse background.
0C0C	006014	SSC, 3, 2	Access the red and green channels.
240A	022012	LE1 (10)	These 4 instructions describe boundaries of the rectangle.
283C	024074	LE2 (60)	
3414	032024	LL1 (20)	
303C	030074	LLR (60) or LL2 (60) LL2 (60)	This instruction completes the description of the rectangle and triggers execution of the operation -- namely the writing of the "black" box at the specified location in both Ch 3 and Ch 2 simultaneously, thus erasing the vector and its labels.